Computational Management Science (2025) 22:3
https://doi.org/10.1007/s10287-025-00529-2

ORIGINAL PAPER

University course timetabling with multi-section courses, room stability and lecturer preferences: an application in a business school

Akin Ozkan[1] · **Aydin Ulucan**[2] · **Ceren Dirik**[3] · **Kazim Baris Atici**[2]

Received: 30 July 2024 / Accepted: 5 January 2025 / Published online: 24 February 2025
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2025

Abstract
Education timetabling models are widely applied approaches for the effective planning of university programs all over the world at different levels. The current research deals with a real-world university course timetabling problem at the faculty level with three main features handled simultaneously in a single integer programming model: multi-section courses, room stability and lecturer preferences. An approach to modeling multi-section courses is presented so that at least one section of a mandatory multi-section course can be chosen, both within the same curriculum and across consecutive curriculums. Moreover, room stability constraints, which have been only addressed in two-stage solutions in previous studies, are integrated to ensure that courses of a student group are assigned to the same classrooms as much as possible. Maximizing the satisfaction level of the lecturers by meeting the time slot requests is also taken into consideration. After the proposed model is introduced, the solution to the model is obtained without encountering out-of-memory errors by extending the Daskalaki et al. (Eur J Oper Res 153(1): 117–135, Daskalaki et al., Eur J Oper Res 153:117–135, 2004) approach for reducing the solution space for a large-scale problem. The computational experiments at a large-scale faculty with real-world data show that the proposed model solves the problems efficiently. The proposed model can be adapted to problems with similar structures and easily solved with existing optimization solvers, even for large faculties or departments.

Keywords Integer programming · University course timetabling · Multi-section courses · Room stability

1 Introduction

For over 50 years, the Operations Research community has been interested in timetabling problems such as nurse rostering, sports scheduling, transportation planning, and educational timetabling (Burke et al. 2007). Among these, educational

Extended author information available on the last page of the article

timetabling stands out as one of the most popular research areas. There are two main types of educational timetabling problems as high school and university timetabling, which are further subdivided into course and exam timetabling. Course timetabling belongs to the NP-complete class of problems that are difficult to solve optimally and efficiently (Even et al. 1975; Cooper and Kingston 1995).

University course timetabling problems are generally considered at faculty or department levels. At first glance, it may seem logical to deal with each department's course timetabling problem separately for ease of solution. However, creating completely independent course schedules for faculty departments with shared courses, classrooms, and lecturers may cause several practical problems. For a faculty, there would be cases in which the course timetables of departments should be thought together to avoid potential conflicts and to create feasible schedules. Saying that it is possible to come up with different approaches that are based on splitting the problem into pieces for solving course timetabling problems at the faculty level. One approach is to create timetables for each department one by one, starting with any department. The main drawback of this approach is that it becomes increasingly difficult to establish timetables as fewer and fewer resources are available for each department. Moreover, if a feasible solution cannot be obtained for a department at any point in the resolution process, the timetables of other departments created previously may need to be revised. Another approach would be creating timetables after the shared faculty resources (i.e., classrooms and laboratories) are allocated across departments. In this case, infeasible timetables may be obtained due to the inability to allocate resources optimally, necessitating repeated revision of resource allocation to arrive at a feasible solution. Even if a feasible solution is achieved, many soft constraints may not be satisfied. Besides, since the university course timetabling problem varies according to the unique characteristics of each institution, there is no specific problem structure. Because of the complicated requirements of faculties and departments, the course timetabling problems can be even more complex. Solving this problem with the abovementioned approaches is not only labor-intensive and time-consuming but also often does not yield satisfactory results. In short, to improve the solution quality and shorten the solution time, it is a good idea to address the university course timetabling problem with a complete model rather than its pieces.

This research aims to develop and solve a faculty-wide university course timetabling problem of three main features handled simultaneously: multi-section courses, room stability and lecturer preferences. The first main characteristic of the proposed integer programming model is the handling of mandatory courses with more than one section. We aim to ensure a minimum overlap of the mandatory *multi-section courses* so that a student group can choose between one section of a mandatory multi-section course (we refer to each class of students as a student group, e.g., 1st, 2nd, 3rd, and 4th grades in a 4-year faculty. In this respect, curriculum refers to the courses each student group should take in the relevant semester). The motivation behind this effort is the necessity of no students in a year being left out for taking their mandatory courses due to the lack of capacity. Unlike the previous research that deals with sectioning or multi-section courses via dividing courses into an optimal number of sections and assigning lecturers to these sections (e.g., Beyrouthy

et al. 2007; Broek et al. 2009; Müller and Murray 2010; Schindl 2019), we focus on the placement of these courses to the calendar with minimum overlap. Here, the number of sections and the lecturer for each section are predetermined (as parameters) by considering the number of lecturers and students taking the course. The modeling challenge here is to manage both the overlaps between sections of mandatory multi-section courses and the overlaps of these sections with other mandatory courses simultaneously. As a result, we effectively address these complex constraints and provide guideline to similar problems. The second main characteristic of the proposed IP model is *room stability*. This refers to the state that the courses for a student group are assigned to the same classrooms as much as possible. In the previous research dealing with room stability, such constraints have been addressed only in two-stage solutions and in the second stage (e.g., Hao and Benlic 2011; Lach and Lübbecke 2008, 2012; Lemos et al. 2019). In those models, the courses are assigned to the time slots in the first stage and to the classrooms in the second stage. However, this approach may not obtain a global optimal solution for room stability since the problem is solved in two stages. For a better solution, we aim to integrate the room stability constraints into a single-stage model or at least into a single-stage IP model. The third and final characteristic of the proposed model is associated with *lecturer preferences*. In the majority of the studies on course timetabling problems, attempts are made to increase lecturer satisfaction levels using matrices created by scoring days of the week, time slots, or classroom preferences. In our proposed model, the lecturers choose one or more time slots for their courses, and our model tries to meet these preferences as much as possible. Since there is no requirement to request all courses, this approach is more appropriate as it prevents overloading the model with too much data.

The integration of the abovementioned features to a single-stage IP model at a faculty-level timetabling has been done in a real-world case that covers a faculty of economics and business in a public university in Turkey. The model contributes by introducing all mentioned features *simultaneously* and solving under different problem instances. The solution approach to the proposed IP model is based on extending Daskalaki et al. (2004)'s approach. In the IP model of Daskalaki et al. (2004), the problem's solution space can be reduced by dividing the data into subsets according to indices (i.e., course, time slot, day, classroom, curriculum, and lecturer). We follow the contribution of Daskalaki et al. (2004) and dramatically reduce the number of decision variables by eliminating redundant ones during defining the sets of parameters in our proposed model. Instead of handling all potential decision variables, the solution space is reduced beforehand by using subsets (e.g., if a lecturer does not teach any courses for a specific student group, associated variables have not been defined). Thus, we show that our course timetabling problem with multi-section courses, room stability, and preferences can be solved without encountering out-of-memory errors. As given in Al-Yakoob and Sherali (2006, 2007), it is hard to solve large-scale course timetabling problems due to out-of-memory errors. Therefore, the approach used in this study is also expected to help overcome this issue. The proposed model and solution approach to this model can be adapted to problems with similar structures and can easily be solved with existing optimization solvers, even for large faculties or departments.

The remainder of the paper is organized as follows: Sect. 2 describes typical characteristics of the university course timetabling problems and briefly reviews the course timetabling problems, focusing on IP formulations. The problem definition is presented in Sect. 3. Mathematical models are given in Sect. 4, and computational results are provided in Sect. 5. Section 6 summarizes the study and concludes with suggestions for future research.

2 University course timetabling problem and related research

The university course timetabling problem is a combinatorial optimization problem that falls under the educational timetabling classification along with high school timetabling problems. Both types of problems are further subdivided into two main categories: course timetabling and exam timetabling (Schaerf 1999). These problems are complex and subject to many constraints, often classified as hard and soft constraints (e.g., Burke and Petrovic 2002; Birbas et al. 2009; Babaei et al. 2015; Vermuyten et al. 2016). Hard constraints are those that must be met, e.g., assigning a lecturer to only one classroom and one course in a time slot. Soft constraints, on the other hand, refer to desirable but not obligatory constraints, e.g., some elective courses should have minimum overlap.

The university course timetabling problem is divided into two categories as post-enrollment-based and curriculum-based course timetabling problem. In the former problem, as its name signifies, enrollment information of students is known and used to avoid timetable overlaps (e.g., Ceschia et al. 2012; Goh et al. 2017). In the latter problem, course enrollment is usually done after the course timetabling is created (e.g., Daskalaki and Birbas 2005; Lach and Lübbecke 2012; Müller and Rudová, 2016; Kiefer et al. 2017). In this problem structure, which student will take which course is not known in advance. For this reason, the courses of student groups are tried to be assigned in a way that there is no overlap. However, students may request to take some courses from a different curriculum. This situation usually happens for the curriculum of the previous or next class (consecutive curriculum). In such cases, extra constraints can be added to the model to ensure that these courses do not overlap or overlap minimally. In this study, we address the curriculum-based university course timetabling problem.

Due to its non-specific nature and NP-completeness, different approaches have been used in the literature to solve university course timetabling problems. Feizi-Derakhshi et al. (2012) and Babaei et al. (2015) classify these approaches into four categories. The first category includes operations research techniques such as graph coloring (Welsh and Powell 1967; de Werra 1997; Dandashi and Al-Mouhamed 2010), linear programming (LP), and IP (Dimopoulou and Miliotis 2001, 2004; Daskalaki et al. 2004; Daskalaki and Birbas 2005; Al-Yakoob and Sherali 2006, 2007), goal programming (Badri et al. 1998), and constraint satisfaction programming (Kang and White 1992; Deris et al. 2000; Zhang and Lau 2005). The second category includes meta-heuristics such as tabu search (Alvarez-Valdes et al. 2002; Aladag et al. 2009; Hao and Benlic 2011) and genetic algorithm (Deris et al. 1999; Wang 2002). The third category includes multi-criteria and multi-objective

techniques (Jamili et al. 2018). The final category includes modern intelligent methods such as fuzzy theory (Asmuni et al. 2005; Chaudhuri and De 2010), hybrid methods (Abdullah et al. 2007; Kohshori and Abadeh 2012; Badoni et al. 2014), and artificial intelligence-based approaches (Lai et al. 2008).

Among the approaches mentioned above, IP models should be designed efficiently for university course timetabling problems. Otherwise, IP-modeled problems may not be solved or require a lot of computer time, especially for large-scale timetabling problems (MirHassani 2006). Therefore, IP is often used in a hybrid form with other approaches (Babaei et al. 2015). However, with hardware and software advances, IP models are once again a viable approach for many combinatorial problems (MirHassani and Habibi 2013).

In the literature, several studies try to solve the course timetabling problem with the IP method. For example, de Werra (1985) proposed a simple linear model for the university course timetabling problem but did not include situations that may be encountered in real-life applications. Daskalaki et al. (2004) and Bakır and Aksop (2008) developed single-stage IP models to solve course timetabling problems, which are real-world applications. As we do in our study, these models are created by subdividing the data according to indices (i.e., course, time slot, day, classroom, curriculum, and lecturer). Similarly, MirHassani (2006) suggested an IP model for solving a real-life course timetabling problem and reduced the solution space by indirectly adding classroom assignments and student requests to the model. Al-Yakoob and Sherali (2006) proposed single-stage and two-stage linear models to solve various course timetabling problems where the time slots to assign courses are predetermined according to the university policy, and the requests of lecturers and students are taken into account. The authors could not solve some problems due to out-of-memory errors, especially for the single-stage model, and obtained only near-optimal solutions for other problems. Al-Yakoob and Sherali (2007) developed an IP model that considers traffic density and gender policy in the university course timetabling problem. For some scenarios, the solutions continued until an out-of-memory error was encountered, and then the solution times and distances from the optimal solution were recorded. Schimmelpfeng and Helber (2007) suggested an IP model that can solve the course timetabling problem as a whole. Their model is able to come up with a solution within minutes for a problem with approximately 150 courses, 100 lecturers, and 650 students. Phillips et al. (2015) put forward IP models for course timetabling problems by first assigning courses to time slots and then assigning these courses to classrooms. The purpose of approaching the problem in two phases is to enable large-scale complex university course timetabling problems to be solved with IP models. In addition, the university course timetabling study by Phillips et al. (2017) addressed an IP model for the minimal perturbation in a real-data problem. Bagger et al. (2019) presented an IP relaxation model for a course timetabling problem to minimize violations of soft constraints like room stability. The model is tested on some real-world problems using the *Gurobi Solver*. Lemos et al. (2019) proposed a two-stage integer linear programming model for classroom usage optimization. The first stage aims to maximize student capacity for each classroom, whereas the second stage minimizes classroom changes for the students. Algethami and Laesanklang (2021) designed a multi-objective mixed-integer

programming model for a post-enrolment-based course timetabling problem at the faculty level. This model is also tested on different real-world instances.

3 Problem definition

A course timetabling problem in the faculty of economics and administrative sciences of a public university in Turkey is used as an illustrative example of the proposed approach.

The basic structure of the university course timetabling problem consists of day, time slot, student group, lecturer, course, and classroom dimensions. These dimensions can be defined for our real-life course timetabling problem as follows:

- All courses are three hours long and are taught in a single block. For daytime education, a day is divided into two time slots: morning and afternoon.
- Undergraduate programs are four years long, and every grade represents a group of students. In addition, which course will be taught by which lecturer is predetermined at the department board meetings. Some lecturers teach in more than one department.
- The courses in the curriculums are divided into mandatory and elective courses. Mandatory courses are courses that must be taken to graduate from a program. Elective courses, on the other hand, are courses that students generally choose from a particular group of courses according to their interests to complete the minimum credits required for graduation. Additionally, there are mandatory/elective multi-section courses. These courses have the same content and are given by the same or different lecturers. Some mandatory multi-section courses are also common to the student groups across multiple departments in our real-world problem.
- Courses requiring special equipment are given in the laboratory, others in regular classrooms, and all faculty classrooms are open for use in all departments. In addition, special equipment courses are designed as multi-section courses to ensure that groups of students can fit into one laboratory.

3.1 Hard constraints

The constraints that must be met are as follows:

Hard Constraint 1. A lecturer must be assigned to only one classroom at a time slot.

Hard Constraint 2. Lecturers must not be assigned to time slots when they are unavailable.

Hard Constraint 3. Courses must be assigned to classrooms that meet their capacity and specific requirements.

Hard Constraint 4. More than one course must not be assigned to a classroom in the same time slot.

Hard Constraint 5. All courses in the curriculums must be assigned to the timetable.

Hard Constraint 6. Mandatory courses in the curriculum of a student group must not be assigned to the same time slot.

Hard Constraint 7. There are some common courses that students of more than one department follow. The time slots and classrooms these courses will be assigned are known. For this reason, other courses of a student group taking common courses should not be assigned to the time slots and classrooms of the common courses.

Hard Constraint 8. A mandatory multi-section course in the curriculum of a student group should be able to be assigned to the same time slot, unlike mandatory non-multi-section courses. However, this flexibility should allow students to choose at least one section of a multi-section course.

Hard Constraint 9. Elective courses in the curriculum of a student group must not overlap with the mandatory courses in the same curriculum. Likewise, if all sections of a mandatory multi-section course of a student group are assigned to the same time slot, then none of the elective courses in the curriculum of this student group should be assigned to this time slot.

3.2 Soft constraints

The constraints that are desired but not obligatory are as follows:

Soft Constraint 1. Mandatory courses of consecutive curriculums should overlap as little as possible. As undergraduate programs are four years long, the consecutive curriculums are 1st–2nd, 2nd–3rd, and 3rd–4th grades, respectively.

Soft Constraint 2. Mandatory multi-section courses of consecutive curriculums should overlap as little as possible. In other words, these courses

Table 1 Information on departments

Department ID	Name of department	NL	Number of courses				
			M	E	MS	ES	Total
MAN	Business administration	29	21	27	16	–	48
ECO	Economics (in English)	19	17	10	4	–	27
EKO	Economics (in Turkish)	23	25	11	16	–	36
KAY	Political science and public administration	29	9	25	–	–	34
MAB	Public finance	25	17	20	7	2	37
SAI	Health management	18	13	11	–	–	24
INR	International relations	16	16	10	–	–	26
ATB	Family and consumer Sciences	21	17	11	–	2	28
SHY	Social work	22	29	16	29	–	45
Total		202	164	141	72	4	305

NL: number of lecturers, *M*: mandatory, *E*: elective, MS: mandatory multi-selection, *ES*: elective multi-section

Table 2 Problems

Problems	Departments
Problem 1	MAN + ECO + EKO + KAY + MAB + SAI + INR + ATB + SHY
Problem 2	MAN + ECO + EKO + KAY + MAB + SAI + INR + ATB
Problem 3	MAN + ECO + EKO + KAY + MAB + SAI + INR
Problem 4	MAN + ECO + EKO + KAY + MAB + SAI
Problem 5	MAN + ECO + EKO + KAY + MAB
Problem 6	MAN + ECO + EKO + KAY
Problem 7	MAN + ECO + EKO
Problem 8	MAN + ECO
Problem 9	MAN

should be assigned in such a way that students can choose at least one section of these courses.

Soft Constraint 3. Elective courses in the curriculum of student groups should not be assigned to the same time slot as much as possible.

Soft Constraint 4. If lecturers request time slots for the assignment of their courses, these requests should be met as much as possible.

Soft Constraint 5. The room stability constraint set ensures that the courses of the student groups are assigned to the same classrooms as much as possible.

Soft Constraint 4 aims to increase the satisfaction levels of lecturers. The remaining soft constraints (Soft Constraints 1, 2, 3, and 5) concern the students. In some faculties, the above constraints may be considered mandatory constraints. However, for our real-life problem, these constraints should be regarded as soft constraints due to lacking human resources and physical space.

3.3 Data

In our real-life problem, the faculty consists of nine departments. Information on the number of courses and lecturers in these departments can be seen in Table 1. There are a total of 305 courses in the spring semester curriculum. 16 lecturers teach in more than one department. For this reason, although the total number of lecturers appears to be 202, there are 184 lecturers. The faculty has 35 classrooms with different capacities, including two laboratories. All classrooms are in the same building. Some courses and some lecturers are common between departments.

The main problem in this study is a faculty course timetabling problem (Problem 1). However, to better observe our models' performance, eight problems (Problems 2–9) are created by removing one department at a time. These problems can be seen in Table 2.

4 Exact formulation

The notation used in our models is similar to that of Daskalaki et al. (2004). The sets, parameters, and decision variables for the problem are defined in the following sub-headings.

4.1 Sets and parameters

- The set of time slots in a week to which a course can be assigned is indicated by I. Since each day consists of two time slots (morning and afternoon), the time slot set is $I = \{1, 2, \ldots, 10\}$
- The set of curriculums (student groups) is indicated by K. Thus, the curriculum set of nine fourth-grade departments is $K = \{1, 2, \ldots, 35\}$
- The set of lecturers is indicated by L, $L = \{1, 2, \ldots, 184\}$
- The set of courses is indicated by M, $M = \{1, 2, \ldots, 305\}$
- The set of classrooms is indicated by N, $N = \{1, 2, \ldots, 35\}$

The parameter sets are defined according to the basic indices: *I, K, L, M,* and *N.* These parameters are as follows (Table 3).

The construction of parameter sets using indices significantly reduces both the total number of decision variables and the constraints, as well as satisfies many hard constraints without any extra constraints. For example, consider the scenario where indices for student groups k, lecturers l, and courses m iterate over all combinations. Without filtering redundant combinations (as in traditional approaches),

Table 3 Parameters

Notations	Description
K_i	Set of student groups assignable to time slot $i, k \in K$
K_l	Set of student groups taught a course or more by lecturer $l, k \in K$
L_i	Set of lecturers available in time slot $i, l \in L$
L_k	Set of lecturers who teach one course or more for student group $k, l \in L$
L_m	The lecturer who teaches course m (only one lecturer for every course), $m \in M, l \in L$
$K_{li} = K_l \cap K_i, \ldots L_{ki} = L_k \cap L_i, \ldots L_{kim} = L_k \cap L_i \cap L_m$	
M_k	Set of courses offered for student group $k, m \in M$
M_l	Set of courses taught by lecturer $l, m \in M$
M_n	Set of courses suitable for classroom $n, m \in M$
$M_{kl} = M_k \cap M_l, \ldots M_{kln} = M_k \cap M_l \cap M_n$	
N_i	Set of available classrooms during time slot $i, n \in N$
N_{mk}	Set of suitable and available classrooms for student group k and course $m, n \in N$
$N_{mki} = N_{mk} \cap N_i$	
I_n	Set of time slots when classroom n is available, $i \in I$
I_k	Set of time slots when student group k is available, $i \in I$
I_l	Set of time slots when lecturer l is available, $i \in I$
$I_{ln} = I_l \cap I_n, \ldots I_{nk} = I_n \cap I_k, \ldots I_{lnk} = I_l \cap I_n \cap I_k$	
$c \in C$	Set of mandatory multi-section courses
C_k	Set of mandatory multi-section courses offered for student groups $k, c \in C$
S_{kc}	Set of all sections of mandatory course c offered for student group $k, s \in S$
M_k^{sec}	Set of all sections of each course in $C_k, m \in M$
M_{kl}^{man}	Set of mandatory non-multi-section courses taught by lecturer l to student group $k, m \in M$
M_k^{elec}	Set of elective courses offered for student group $k, m \in M$
M_{kl}^{elec}	Set of elective courses taught by lecturer l to student group $k, m \in M$
I_m^{req}	Set of time slots requested for the assignment of course $m, i \in I$
M_{kl}^{req}	Set of courses taught by lecturer l to student group k, containing time slot requests, $m \in M$
KK_i	Set of consecutive pairs of student groups (curriculums) available in time slot $i, (k_a, k_b) \in K$
a_k	Total number of courses offered for student group k
b_k	The number of mandatory multi-section courses in C_k
r_{kc}	The number of sections of mandatory multi-section course c offered for student group $k, c \in C$
say_t	The number of courses with time slot requests
$req_c = 1$	The unit cost not meeting each request

PRA ensures course m taught by lecturer l to student group k is assigned to desired classroom n and time slot i; that is, $x_{i,k,l,m,n} = 1, (i, k, l, m, n) \in IxKxLxMxN$

the total number of decisions would equal the Cartesian product of the indices $KxLxM$, resulting in $35 \times 184 \times 305 = 1{,}964{,}200$ variables. However, by constructing sets like K, L_k, and M_{kl} (as in our problem), we eliminate redundant combinations where, for instance, a lecturer l does not teach any courses for a specific student group k. Similarly, courses m unrelated to a specific k and l pairs are excluded. This way, the number of decision variables is reduced to just 305, which also

corresponds to the actual data from our problem, representing the valid combinations $(k, l, m) \in \left(\forall k \in K, \forall l \in L_k, \forall m \in M_{kl}\right)$. This approach effectively reduces the memory usage in a computer's RAM by excluding redundant combinations upfront and makes the modeling process more practical.

4.2 Decision variables

In our models, decision variables are generated according to the sets created using indices. In this way, the total number of decision variables is significantly reduced by excluding redundant variables that are not included in the solution. The change in the number of decision variables in our problem is as follows:

Number of Decision Variables in the Real Case Problem

$x_{i,k,l,m,n}(i, k, l, m, n) \in IxKxLxMxN \longrightarrow$ 824,964,000

$x_{i,k,l,m,n}(i, k, l, m, n) \in \left(\forall i \in I, \forall k \in K_i, \forall l \in L_{ki}, \forall m \in M_{kl}, \forall n \in N_{mki}\right) \longrightarrow$ 53,021

As seen above, if the parameters are not formed using the indices, the number of decision variables resulting from the product of the parameter sets can reach vast numbers. Thus, an out-of-memory error is often received. In other words, the algorithm cannot start solving the problem because there is insufficient memory in RAM for the decision variables. In similar studies, two-stage or hybrid models are usually designed to overcome the out-of-memory error. In our study, on the other hand, the course timetabling problem can be solved with a single-stage linear programming model by significantly reducing the number of decision variables, as explained in the previous section. In order to achieve this, we utilize *list comprehension*[1] in Python for sub-sets creation during the implementation phase. This approach simplifies the creation of complex parameter sets by filtering and combining values directly within the *list comprehension*. As a result, redundant combinations are filtered out early in the process.

The decision variables are defined as follows:

$x_{i,k,l,m,n}$ 1 if course m given by lecturer l to student group k is scheduled on time slot i and classroom n; 0 otherwise, $(i, k, l, m, n) \in \left(\forall i \in I, \forall k \in K_i, \forall l \in L_{ki}, \forall m \in M_{kl}, \forall n \in N_{mki}\right)$

$y_{i,k}$ This integer decision variable denotes the number of overlaps on the time slot i for student group $k, (i, k) \in \left(i \in I, k \in K_i\right)$. It's created to ensure the minimum number of overlaps for a student group's elective courses (for Soft Constraint 3)

$Z^{man}_{i,(k_a,k_b)}$ This integer decision variable denotes the number of overlaps on the time slot i for student group k_a and k_b, $\left(i, (k_a, k_b)\right) \in \left(i \in I, (k_a, k_b) \in KK_i\right)$. It's created to

[1] List comprehensions in Python: https://docs.python.org/3/tutorial/datastructures.html#list-comprehensions.

ensure the minimum number of overlaps for the mandatory courses of consecutive curriculums (for Soft Constraint 1)

$Z^{sec}_{i,(k_a,k_b)}$ Since the procedure for mandatory multi-section courses is different, this integer decision variable is also created for these courses, $(i,(k_a,k_b) \in (i \in I,(k_a,k_b) \in KK_i)$. It ensures the minimum number of overlaps for the mandatory multi-section courses of consecutive curriculums (for Soft Constraint 2)

4.3 Constraints

The constraints of our models are given below.

$$\sum_{k\in K_{li}} \sum_{m\in M_{kl}} \sum_{n\in N_{mki}} x_{i,k,l,m,n} \le 1 \quad \forall i \in I, \forall l \in L_i \tag{1}$$

$$\sum_{l\in L_{ki}} \sum_{m\in M^{man}_{kl}} \sum_{n\in N_{mki}} x_{i,k,l,m,n} + \sum_{l\in L_{kim_s}} \sum_{n\in N_{m_s ki}} x_{i,k,l,m_s,n} \le 1 \quad \forall i \in I,\ \forall k \in K_i,\ \forall m_s \in M^{\text{sec}}_k \tag{2}$$

$$\sum_{l\in L_{ki}} \sum_{m\in M^{man}_{kl}} \sum_{n\in N_{mki}} x_{i,k,l,m,n} + \sum_{l\in L_{kim_s}} \sum_{n\in N_{m_s ki}} x_{i,k,l,m_s,n} \le 1 \quad \forall i \in I,\ \forall k \in K_i,\ \forall m_s \in M^{elec}_k \tag{3}$$

$$\sum_{m\in S_{kC_{k[z]}}} \sum_{l\in L_{kim}} \sum_{n\in N_{mki}} x_{i,k,l,m,n} + \sum_{l\in L_{kim_s}} \sum_{n\in N_{m_s ki}} x_{i,k,l,m_s,n} \le r_{kC_{k[z]}} \quad \forall i \in I,\ \forall k \in K_i,\ \forall z \in \left[1,\ldots,b_k\right],\ m_s \in M_k,\ m_s \backslash S_{kC_{k[z]}} \tag{4}$$

$$\sum_{l\in L_{ki}} \sum_{m\in M^{elec}_{kl}} \sum_{n\in N_{mki}} x_{i,k,l,m,n} \le 1 + y_{i,k} \quad \forall i \in I,\ \forall k \in K_i \tag{5}$$

$$\sum_{d\in\left[k_a,k_b\right]} \sum_{l\in L_{di}} \sum_{m\in M^{man}_{dl}} \sum_{n\in N_{mdi}} x_{i,d,l,m,n} \le 1 + Z^{man}_{i,(k_a,k_b)} \quad \forall i \in I,\ \forall (k_a,k_b) \in KK_i \tag{6}$$

$$\sum_{m\in S_{k_1 C_{k_1[z]}}} \sum_{l\in L_{k_1 im}} \sum_{n\in N_{mk_1 i}} x_{i,k_1,l,m,n} + \sum_{l\in L_{k_2 im_s}} \sum_{n\in N_{m_s k_2 i}} x_{i,k_2,l,m_s,n} \le r_{k_1 C_{k_1[z]}} + Z^{sec}_{i,(k_a,k_b)} \quad \forall i \in I,\ \forall (k_a,k_b) \in KK_i, \quad \forall k_1 \in [k_a,k_b],\ \forall z \in \left[1,\ldots,b_{k_1}\right],\ \forall k_2 \in [k_a,k_b],\ \forall m_s \in M^{sec}_{k_2},\ k_1 \ne k_2 \tag{7}$$

$$\sum_{k\in K_i} \sum_{l\in L_{ki}} \sum_{m\in M_{kln}} x_{i,k,l,m,n} \le 1 \quad \forall n \in N,\ \forall i \in I_n \tag{8}$$

$$\sum_{l\in L_k}\sum_{m\in M_{kl}}\sum_{n\in N_{mk}}\sum_{i\in I_{lnk}} x_{i,k,l,m,n} = a_k \quad \forall k \in K \tag{9}$$

$$\sum_{n\in N_{mk}}\sum_{i\in I_{lnk}} x_{i,k,l,m,n} = 1 \quad \forall k \in K,\ \ \forall l \in L_k,\ \ \forall m \in M_{kl} \tag{10}$$

Constraint (1) ensures that only one student group, one classroom, and one course can be assigned to any lecturer in any time slot. Constraint (2) states that at most one course can be assigned to student group k in time slot i. More precisely, for student group k, the first part of this constraint prevents the overlapping of mandatory courses, and the second part prevents the overlapping of mandatory multi-section courses with mandatory courses. In addition, this constraint allows assigning sections of mandatory multi-section courses to the same time slot. The first part of Constraint (3) is the same as the first part of Constraint (2). The second part of this constraint prevents overlapping of elective and mandatory non-multi-section courses of student group k. If all sections of a student group's mandatory multi-section course are assigned to the same time slot, Constraint (4) ensures that none of the other courses in the curriculum of this student group are assigned to this time slot. $C_{k[z]}$ denotes the zth ranked mandatory multi-section course in C_k. Constraint (5) is a soft constraint that ensures the minimum overlap of elective courses for student groups by minimizing the decision variable $y_{i,k}, (i,k) \in \left(i \in I, k \in K_i\right)$ in the objective function. Similarly, Constraints (6) and (7) are soft constraints for mandatory courses and mandatory multi-section courses of consecutive curriculums, respectively. Constraint (6) ensures the minimum overlap of mandatory courses of consecutive student groups by minimizing decision variable $Z^{man}_{i,(k_a,k_b)}, \left(i,(k_a,k_b)\right) \in \left(i \in I, (k_a,k_b) \in KK_i\right)$ in the objective function. Constraint (7) ensures the minimum overlap of mandatory multi-section courses of consecutive student groups by minimizing decision variable $Z^{sec}_{i,(k_a,k_b)}, \left(i,(k_a,k_b\right) \in \left(i \in I, (k_a,k_b\right) \in KK_i)$ in the objective function. Constraint (8) states that only one student group, one lecturer, and one course can be assigned to any classroom at any time slot. Finally, Constraint (9) ensures that all courses in the curriculum of each student group are assigned, while Constraint (10) states that each course is assigned only once.

4.4 Objective function

The objective function in this study consists of four parts, as seen in Eq. (11). In this objective function, the first part minimizes the total number of overlapping elective courses for any curriculum, the second and third parts minimize the total number of overlapping mandatory and mandatory multi-section courses for consecutive curriculums, and the last part maximizes the level of realization of course assignment preferences. There may be one or more time slot preferences for every course, and this model tries to ensure that as many preferences as possible are met. As can be seen, the first three parts of the objective function aim to increase student satisfaction, while the fourth part aims to increase lecturer satisfaction. If the importance of these parts is different, their realization priorities can be changed by determining a

cost coefficient for each part. In our problem, the coefficient of realizing the lecturers' preferences is set as 1 ($req_c = 1$).

$$MinZ = \left\{ \begin{array}{l} \sum_{i \in I} \sum_{k \in K_i} y_{i,k} \\ \quad + \sum_{i \in I} \sum_{(k_a,k_b) \in KK_i} Z^{man}_{i,(k_a,k_b)} \\ \quad + \sum_{i \in I} \sum_{(k_a,k_b) \in KK_i} Z^{sec}_{i,(k_a,k_b)} \\ \quad + say_t * req_c - \left(req_c * \sum_{i \in I^{req}_m} \sum_{n \in N_{mki}} x_{i,k,l,m,n} \;\; \forall k \in K, \;\; \forall l \in L_k, \;\; \forall m \in M^{req}_{kl} \right) \end{array} \right\} \tag{11}$$

4.5 Room stability

In this study, the room stability constraint set is added as another soft constraint to the IP model. In fact, in our real-world problem, there is no strict requirement for room stability since all classrooms are in the same building. Nevertheless, it can improve the quality of the solution to this problem. Also, the way room stability is formulated in our one-stage IP model can serve as a guide to adapt it to other problems in similar structures.

To integrate room stability into our model, additional decision variables and additional constraints are created as follows:

4.5.1 Decision variables

- $sum_RS_k, (k \in K)$ is an integer decision variable used in the objective function to minimize the number of different classrooms for student group k. Since different types of classrooms are needed for the courses in each student group, the lower bound of this variable is determined as 4 to reduce solution times. This value indicates the possible number of classroom types required for each student group.
- $sum_RS_{kn}, (k \in K, n \in N)$ is the number of courses assigned to classroom n for student group k.
- $RS_{kn}, (k \in K, n \in N)$ is a binary variable that takes the value 1 if any course of student group k is assigned to classroom n; otherwise, it takes the value 0.
- $comp_{kn}, (k \in K, n \in N)$ is a continuous variable that can only take a value between [0–1).

4.5.2 Constraints

$$\sum_{i \in I_{nk}} \sum_{l \in L_{ki}} \sum_{m \in M_{kln}} x_{i,k,l,m,n} = sum_RS_{kn} \quad \forall k \in K, \;\; \forall n \in N \tag{12}$$

$$\frac{sum_RS_{kn}}{a_k} + comp_{kn} = RS_{kn} \quad \forall k \in K, \;\; \forall n \in N \tag{13}$$

$$\sum_{n \in N} RS_{kn} \leq sum_RS_k \quad \forall k \in K \tag{14}$$

4.5.3 Objective function

$$Equation\,(11) + \sum_{k \in K} sum_RS_k \tag{15}$$

Constraint (12) calculates the total number of courses assigned to classroom n for student group k. Constraint (13) ensures that RS_{kn} takes the value of 1 if at least one course of student group k is assigned to classroom n. Constraint (14) allows minimizing the total number of different classrooms for each student group. For room stability, Constraints (12), (13), and (14) should be added to the Constraints (1) – (10). In addition, a new IP model that considers room stability is created by replacing Eq. (11) with Eq. (15).

5 Computational results

In order to implement the proposed models, an input file in the.ecct format was first created. This file consists of courses, rooms, teacher unavailable, room_unavailable, curriculum_un_assign, and requests. Under the courses heading, there are the core name of the course, the name of the course (ID), the department of the course, the lecturer of the course, the number of students taking the course, the status of the course (mandatory: 0, elective: 1), the section of the course (0 if there is no section, the section number otherwise), the curriculum of the course (each number represents a different curriculum), and the classroom type of the course (0 regular, 1 laboratory). The room heading consists of the name (ID), capacity, and type of each classroom. The teacher_unavailable heading contains the name of a lecturer and the day and time slot when this lecturer is unavailable. The room_unavailable heading contains the name of a classroom and the day and time slot when this classroom is unavailable. The curriculum_un_assign heading indicates the day and time slot when the curriculum (student group) must not be assigned. The requests heading indicates the day and time slot when the course is requested to be assigned (see "Appendix" for an example of the input file). This input file makes it easier to add extra data and new features. The generation of the parameters' sets is executed through this input file using Python. For the solution of the mathematical models, Gurobi version 9.5.1[2] is used as a solver on a PC processor equipped with Intel Core i3 at 3.60 GHz and 8 GB RAM.

9 problems (see Table 2) with different sizes are solved to evaluate the proposed IP models. The problem sizes are given in Table 4, and solution times and cost values are summarized in Table 5.

The standard model contains a minimum of 1448 and a maximum of 10,223 equations, a minimum of 10,083 and a maximum of 53,770 variables, while the nonzeros of this model range from a minimum of 107,126 to a maximum of 656,530.

[2] Gurobi Optimization https://www.gurobi.com/.

Table 4 Size of the problems

	Problem Sizes			Standard Model Sizes			Room Stability Model Sizes		
	# of curriculums	# of courses	# of lecturers	# of rows	# of columns	# of non-zeros	# of rows	# of columns	# of non-zeros
Problem 1	35	305	184	10,223	53,770	656,530	13,198	58,215	712,846
Problem 2	32	260	162	6631	46,480	439,064	9351	50,544	488,072
Problem 3	28	232	142	6232	44,010	421,568	8612	47,566	467,590
Problem 4	24	206	126	5886	39,357	392,210	7926	42,405	433,263
Problem 5	20	182	110	5535	35,168	364,497	7235	37,708	401,124
Problem 6	16	145	89	4696	26,263	282,516	6056	28,295	310,206
Problem 7	12	111	60	4074	18,735	236,759	5094	20,259	257,252
Problem 8	8	75	48	1688	13,783	126,429	3213	14,875	179,716
Problem 9	4	48	29	1448	10,083	107,126	2385	10,638	147,035

Table 5 Solution times and cost values of the problems

Solution time/cost value	Problems								
	#1	#2	#3	#4	#5	#6	#7	#8	#9
Standard model	80.12/10	24.68/10	18.97/9	15.44/9	31.45/9	2.5/8	3.52/8	1.7/8	2.53/8
Room stability model	270.65/10	57.47/10	28.36/9	24.54/9	51.13/9	7.03/8	6.05/8	4.05/8	2.95/8

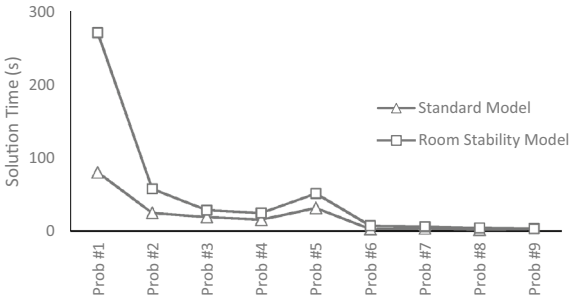


Fig. 1 Solution times for models

The room stability model, on the other hand, involves 2345–13,198 equations, 10,638–58,215 variables, and 147,035–712,846 non-zeros, representing minimum and maximum values, respectively.

The computation of the optimal timetables of Problem 1, which is our main problem, requires approximately 80 s for the standard model and 271 s for the room stability model. Additionally, even though the room stability model allows student groups to be assigned to the same classes as much as possible, the solution times are longer than in the standard model. This situation can be seen in Table 5 and Fig. 1. In this context, it should be emphasized that the solution times for larger and more complex problems will be even longer.

6 Conclusions

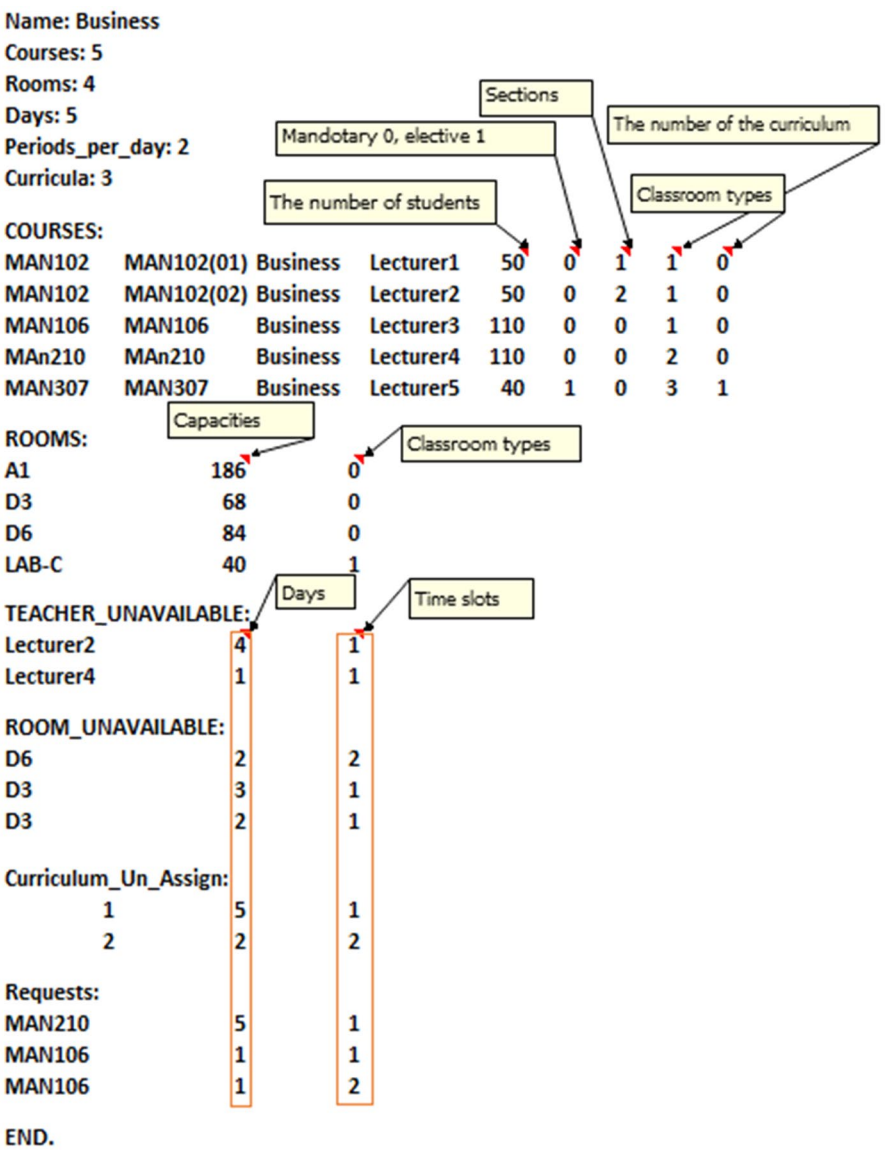
The current research presents a comprehensive integer programming model that effectively addresses a real-world university course timetabling problem at the faculty level. The model successfully incorporates three key features simultaneously: multi-section courses, room stability, and lecturer preferences. By introducing an innovative approach to modeling multi-section courses, the model ensures that students can choose at least one section of a mandatory multi-section course, both within the same curriculum and across consecutive curriculums. Furthermore, the integration of room stability constraints in a single-stage solution marks a significant advancement over previous two-stage solutions, ensuring that courses of a student group are assigned to the same classrooms as much as possible. The model also takes into account the satisfaction level of lecturers by maximizing the fulfillment of their time slot requests.

Since a faculty-level problem is quite large, to overcome the challenge of out-of-memory errors when solving the model, the research extends Daskalaki et al. (2004) approach for reducing the solution space. The computational experiments conducted using real-world data from a large-scale faculty demonstrate the efficiency and effectiveness of the proposed model in solving timetabling problems.

The proposed model is adaptable to problems with similar structures as it can be easily solved using existing optimization solvers, even for large faculties or departments. Our course timetabling models are designed for problems with courses taught in 3-hour blocks but can also be adapted for courses that are more disaggregated on an hourly basis. This research contributes to the field of education timetabling by providing a comprehensive and efficient solution that addresses multiple critical aspects of the problem simultaneously. The model's successful implementation has the potential to significantly improve the planning and management of university programs, ultimately benefiting students, lecturers, and administrators alike.

Appendix

Data input format.



Author contribution Authors confirm that the corresponding author has read the journal policies and submit this manuscript in accordance with those policies. A.O. contributed in the conceptualization, data curation, methodology, analysis, visualization and writing. A.U. contributed in the conceptualization, methodology, supervision and writing. C.D. contributed in data curation, methodology, analysis, visualization and writing. K.B.A. contributed in the conceptualization, methodology, supervision and writing.

Funding No funding was received for conducting this study.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Conflict of Interest The authors declare that the authors have no competing interests as defined by Springer, or other interests that might be perceived to influence the results and/or discussion reported in this paper.

Ethics approval and consent to participate Authors express consent to participate in this article and express that they know, respect and comply with the ethical requirements for the publication of the document.

References

Abdullah S, Burke EK, McCollum B (2007) A hybrid evolutionary approach to the university course timetabling problem. In: 2007 IEEE congress on evolutionary computation (CEC 2007), Singopore

Aladag CH, Hocaoglu G, Basaran MA (2009) The effect of neighborhood structures on tabu search algorithm in solving course timetabling problem. Expert Syst Appl 36(10):12349–12356

Algethami H, Laesanklang W (2021) A mathematical model for course timetabling problem with faculty-course assignment constraints. IEEE Access 9:111666–111682

Alvarez-Valdes R, Crespo E, Tamarit JM (2002) Design and implementation of a course scheduling system using Tabu Search. Eur J Oper Res 137(3):512–523

Al-Yakoob SM, Sherali HD (2006) Mathematical programming models and algorithms for a class–faculty assignment problem. Eur J Oper Res 173(2):488–507

Al-Yakoob SM, Sherali HD (2007) Mixed-integer programming models for an employee scheduling problem with multiple shifts and work locations. Ann Oper Res 155(1):119–142

Asmuni H, Burke EK, Garibaldi JM (2005) Fuzzy multiple heuristic ordering for course timetabling. In: Proceedings of the 2005 UK workshop on computational intelligence (UKCI 2005), London, United Kingdom

Babaei H, Karimpour J, Hadidi A (2015) A survey of approaches for university course timetabling problem. Comput Ind Eng 86:43–59

Badoni RP, Gupta DK, Mishra P (2014) A new hybrid algorithm for university course timetabling problem using events based on groupings of students. Comput Ind Eng 78:12–25

Badri MA, Davis DL, Davis DF, Hollingsworth J (1998) A multi-objective course scheduling model: combining faculty preferences for courses and times. Comput Oper Res 25(4):303–316

Bagger NCF, Desaulniers G, Desrosiers J (2019) Daily course pattern formulation and valid inequalities for the curriculum-based course timetabling problem. J Sched 22:155–172

Bakır MA, Aksop C (2008) A 0–1 integer programming approach to a university timetabling problem. Hacettepe J Math Stat 37(1):41–55

Beyrouthy C, Burke EK, Landa-Silva D, McCollum B, McMullan P, Parkes AJ (2007) The teaching space allocation problem with splitting. In: Burke EK, Rudová H (eds) International conference on the practice and theory of automated timetabling. Springer, pp 228–247

Birbas T, Daskalaki S, Housos E (2009) School timetabling for quality student and teacher schedules. J Sched 12(2):177–197

Broek JVD, Hurkens C, Woeginger G (2009) Timetabling problems at the TU Eindhoven. Eur J Oper Res 196(3):877–885

Burke EK, Petrovic S (2002) Recent research directions in automated timetabling. Eur J Oper Res 140(2):266–280

Burke EK, McCollum B, Meisels A, Petrovic S, Qu R (2007) A graph-based hyper-heuristic for educational timetabling problems. Eur J Oper Res 176(1):177–192

Ceschia S, Gaspero LD, Schaerf A (2012) Design, engineering, and experimental analysis of a simulated annealing approach to the post-enrolment course timetabling problem. Comput Oper Res 39(7):1615–1624

Chaudhuri A, De K (2010) Fuzzy genetic heuristic for university course timetable problem. Int J Adv Soft Comput Appl 2(1):100–123
Cooper TB, Kingston JH (1995) The complexity of timetable construction problems. In: Burke E, Ross P (eds) International conference on the practice and theory of automated timetabling. Springer, pp 281–295
Dandashi A, Al-Mouhamed M (2010) Graph coloring for class scheduling. In: ACS/IEEE international conference on computer systems and applications (AICCSA 2010), Hammamet, Tunisia
Daskalaki S, Birbas T (2005) Efficient solutions for a university timetabling problem through integer programming. Eur J Oper Res 160(1):106–120
Daskalaki S, Birbas T, Housos E (2004) An integer programming formulation for a case study in university timetabling. Eur J Oper Res 153(1):117–135
De Werra D (1985) An introduction to timetabling. Eur J Oper Res 19(2):151–162
De Werra D (1997) The combinatorics of timetabling. Eur J Oper Res 96(3):504–513
Deris S, Omatu S, Ohta H, Saad P (1999) Incorporating constraint propagation in genetic algorithm for university timetable planning. Eng Appl Artif Intell 12(3):241–253
Deris S, Omatu S, Ohta H (2000) Timetable planning using the constraint-based reasoning. Comput Oper Res 27(9):819–840
Dimopoulou M, Miliotis P (2001) Implementation of a university course and examination timetabling system. Eur J Oper Res 130(1):202–213
Dimopoulou M, Miliotis P (2004) An automated university course timetabling system developed in a distributed environment: a case study. Eur J Oper Res 153(1):136–147
Even S, Itai A, Shamir A (1975) On the complexity of time table and multi-commodity flow problems. In: 16th annual symposium on foundations of computer science (pp 184–193). IEEE.
Feizi-Derakhshi MR, Babaei H, Heidarzadeh J (2012) A survey of approaches for university course timetabling problem. In: Proceedings of 8th international symposium on intelligent and manufacturing systems (IMS 2012), Antalya, Turkey
Goh SL, Kendall G, Sabar NR (2017) Improved local search approaches to solve the post enrolment course timetabling problem. Eur J Oper Res 261(1):17–29
Hao JK, Benlic U (2011) Lower bounds for the ITC-2007 curriculum-based course timetabling problem. Eur J Oper Res 212(3):464–472
Jamili A, Hamid M, Gharoun H, Khoshnoudi R (2018) Developing a comprehensive and multi-objective mathematical model for university course timetabling problem: a real case study. In: Proceedings of the international conference on industrial engineering and operations management, Paris, France
Kang L, White GM (1992) A logic approach to the resolution of constraints in timetabling. Eur J Oper Res 61(3):306–317
Kiefer A, Hartl RF, Schnell A (2017) Adaptive large neighborhood search for the curriculum-based course timetabling problem. Ann Oper Res 252(2):255–282
Kohshori MS, Abadeh MS (2012) Hybrid genetic algorithms for university course timetabling. Int J Comput Sci Issues 9(2):446–455
Lach G, Lübbecke ME (2008) Optimal university course timetables and the partial transversal polytope. In: McGeoch CC (ed) International workshop on experimental and efficient algorithms. Springer, pp 235–248
Lach G, Lübbecke ME (2012) Curriculum based course timetabling: new solutions to Udine benchmark instances. Ann Oper Res 194(1):255–272
Lai LF, Wu CC, Hsueh NL, Huang LT, Hwang SF (2008) An artificial intelligence approach to course timetabling. Int J Artif Intell Tools 17(01):223–240
Lemos A, Melo FS, Monteiro PT, Lynce I (2019) Room usage optimization in timetabling: A case study at Universidade de Lisboa. Oper Res Perspect 6:100092
MirHassani SA (2006) A computational approach to enhancing course timetabling with integer programming. Appl Math Comput 175(1):814–822
MirHassani SA, Habibi F (2013) Solution approaches to the course timetabling problem. Artif Intell Rev 39(2):133–149
Müller T, Murray K (2010) Comprehensive approach to student sectioning. Ann Oper Res 181(1):249–269
Müller T, Rudová H (2016) Real-life curriculum-based timetabling with elective courses and course sections. Ann Oper Res 239(1):153–170
Phillips AE, Waterer H, Ehrgott M, Ryan DM (2015) Integer programming methods for large-scale practical classroom assignment problems. Comput Oper Res 53:42–53

Phillips AE, Walker CG, Ehrgott M, Ryan DM (2017) Integer programming for minimal perturbation problems in university course timetabling. Ann Oper Res 252:283–304
Schaerf A (1999) A survey of automated timetabling. Artif Intell Rev 13(2):87–127
Schimmelpfeng K, Helber S (2007) Application of a real-world university-course timetabling model solved by integer programming. OR Spectrum 29(4):783–803
Schindl D (2019) Optimal student sectioning on mandatory courses with various sections numbers. Ann Oper Res 275(1):209–221
Vermuyten H, Lemmens S, Marques I, Beliën J (2016) Developing compact course timetables with optimized student flows. Eur J Oper Res 251(2):651–661
Wang YZ (2002) An application of genetic algorithm methods for teacher assignment problems. Expert Syst Appl 22(4):295–302
Welsh DJA, Powell MB (1967) An upper bound for the chromatic number of a graph and its application to timetabling problems. Comput J 10(1):85–86
Zhang L, Lau S (2005) Constructing university timetable using constraint satisfaction programming approach. In: International conference on computational intelligence for modelling, control and automation and international conference on intelligent agents, web technologies and internet commerce (CIMCA-IAWTIC'06), Vienna, Austria

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Akin Ozkan[1] · Aydin Ulucan[2] · Ceren Dirik[3] · Kazim Baris Atici[2]

✉ Kazim Baris Atici
kba@hacettepe.edu.tr

Akin Ozkan
a.ozkan@alparslan.edu.tr

Aydin Ulucan
aulucan@hacettepe.edu.tr

Ceren Dirik
cerendirik@kku.edu.tr

[1] Department of Business Administration, Mus Alparslan University, Muş, Turkey

[2] Department of Business Administration, Hacettepe University, Ankara, Turkey

[3] Department of Business Administration, Kirikkale University, Kirikkale, Turkey